

CSE 220 Fourier Exam Preparation Guide

Fourier Series, CFT, 2D-CFT, Harmonic Compression, Numerical Verification, and Exam Coding

Updated August 2026

Contents

CSE 220 Fourier Series & Continuous Fourier Transform - Complete Exam Preparation Guide	5
1. What You Need to Be Able to Do	5
2. Pattern Seen in the Previous-Year Questions	5
PYQ 1 - Differentiation Property	5
PYQ 2 - Gaussian Time Shift	6
PYQ 3 - Frequency Shift + Time Scaling	6
PYQ 4 - Parseval	7
PYQ 5 - Identify Sinusoidal Components	7
PYQ 6 - Image Denoising	7
3. Current Offline Assignment - Task 1	7
4. Core FourierEpicycles Code Pattern	8
5. Meaning of an Epicycle Term	9
6. Fourier-Series Energy	9
7. Given Problem - Energy-Preserving Harmonic Pruning	10
Goal	10
8. Energy-Pruning Solution	10
Step 1 - energies	10
Step 2 - total and target	10
Step 3 - sort strongest first	11
Step 4 - index to harmonic number	11
Step 5 - accumulate	11
Step 6 - zero discarded coefficients	11
Step 7 - return	11
9. Reconstruction Error	12
10. Critical Pitfall - Reusing a Pruned Object	12
11. Expected Pruning Trend	13
12. Fourier-Series Properties to Memorize	13
Time shift	13
Time reversal	13
Complex scaling / rotation	14
Spatial translation of the drawing	14
Differentiation	14

Fourier-Series Parseval	15
13. Warm-Up Problem 1 - Top-K Harmonics	15
14. Warm-Up Problem 2 - Fourier-Series Time Shift	15
15. Warm-Up Problem 3 - Velocity Signal	16
16. Warm-Up Problem 4 - Low-Pass Harmonic Reconstruction	16
17. Warm-Up Problem 5 - Coefficient Quantization	17
18. Hard Problem 1 - Symmetric Energy-Constrained Compression	17
19. Hard Problem 2 - Minimum Harmonics for an MSE Constraint	18
20. Hard Problem 3 - Combined Time Shift + Rotation + Scale + Translation	19
21. Hard Problem 4 - Harmonic-Band Detail Enhancement	21
22. Hard Problem 5 - Joint Pruning + Quantization	21
23. Additional Mock Variants Based on This Year's Assignment	22
Variant A - Parameter Shift vs Geometric Translation	23
Variant B - Time Reversal	23
Variant C - Fourier-Series Parseval	23
Variant D - Harmonic Smoothing	24
Variant E - 2D Low-Pass Blurring	24
Variant F - 2D Spatial Shift	24
Variant G - Conjugate Symmetry	24
Variant H - Periodic Stripe-Noise Removal	25
Variant I - Directional Edge Extraction	25
Variant J - 2D Cosine Modulation	25
Variant K - Anisotropic Spatial Scaling	25
Variant L - 2D Parseval	25
24. Continuous Fourier Transform Fundamentals	26
25. When to Use 2π	26
26. Numerical 1D CFT Template	26
27. CFT Properties to Memorize	27
Time shift	27
Frequency shift / modulation	27
Time scaling	27
Time reversal	27
Differentiation	28
Multiplication / convolution	28
Parseval	28
28. Why Time Scaling Gives $1/ a$	28
29. Magnitude and Phase	29
30. Why Phase Near Zero Magnitude Is Unreliable	29
31. Wrapped Phase Error	30
32. Current Offline Assignment - 2D CFT	30

33. Image Array Orientation	31
34. Spatial Frequency Intuition	31
35. Real and Imaginary Parts of the 2D CFT	31
36. Separability	32
37. Loop-Based 2D CFT Pattern	32
38. Magnitude Spectrum	34
39. High-Pass Filtering	34
40. Low-Pass Filtering	35
41. Notch Filtering	35
42. Directional Filtering	35
43. Inverse 2D CFT	36
44. 2D Parseval Numerical Pattern	36
45. NumPy / Python Functions You Need	37
np.linspace	37
np.arange	37
np.abs	37
np.angle	38
np.exp	38
np.trapezoid	38
46. Understanding axis	38
47. np.argsort	39
48. Dictionary Sorting	39
49. [: :-1]	39
50. Boolean Masks	40
51. 2D Masks	40
52. np.meshgrid	40
53. Broadcasting with None	41
54. 3D Broadcasting for Images	41
55. np.mean	42
56. np.gradient	42
57. .copy()	42
58. enumerate	43
59. Complex Parts	43
60. np.hypot	43

61. np.log1p	43
62. Sets and Membership	43
63. Reusable Error Metrics	44
Signal MSE	44
Magnitude MSE	44
Wrapped phase MSE	44
Relative energy error	44
64. Search / Optimization Loop Pattern	45
65. Common Pitfalls	45
Pitfall 1 - Mutating the original coefficient set	45
Pitfall 2 - Forgetting negative harmonics	45
Pitfall 3 - Forgetting $1/T$	45
Pitfall 4 - Wrong exponent sign	46
Pitfall 5 - Time shift vs drawing translation	46
Pitfall 6 - Phase comparison near zero magnitude	46
Pitfall 7 - Plain phase subtraction	46
Pitfall 8 - Wrong image indexing	46
Pitfall 9 - Wrong integration axis	46
Pitfall 10 - Using spatial axes as frequency axes	47
Pitfall 11 - Direct $O(N^4)$ transform	47
Pitfall 12 - FFT when prohibited	47
Pitfall 13 - Re-quantizing or re-amplifying already modified coefficients	47
Pitfall 14 - Losing harmonic identity after sorting	47
Pitfall 15 - Assuming actual energy ratio equals target	47
66. How to Classify a New Exam Question	47
Type A - Property verification	47
Type B - Coefficient processing / compression	48
Type C - Image frequency filtering	48
Type D - Optimization/search	48
67. 30-40 Minute Exam Strategy	48
First 3-5 minutes	48
Next 15-20 minutes	49
Next 5-10 minutes	49
Final 5 minutes	49
68. Minimal Templates to Memorize	49
Fourier-Series coefficient	49
Fourier-Series reconstruction	50
Energy	50
Descending sort	50
MSE	50
Phase wrap	50
Magnitude mask	50
Fresh-object experiment	51
69. Final Formula Sheet	51
Fourier Series	51
CFT using f	52
2D CFT	53
70. Preparation Priority	53
71. Final Self-Test Checklist	54

CSE 220 Fourier Series & Continuous Fourier Transform - Complete Exam Preparation Guide

This guide consolidates the material we have discussed for the CSE 220 Fourier exam: the current of-line assignment, all supplied previous-year online questions, the harmonic-pruning problem, the new practice problems we designed, their solution strategies, background theory, recurring pitfalls, and the Python/NumPy techniques that are likely to be needed in a timed exam.

The recurring exam pattern is: **reuse the offline-assignment framework, add one or two methods, apply a Fourier property or coefficient-processing rule, reconstruct, and verify numerically.**

1. What You Need to Be Able to Do

You should be able to:

1. Compute complex Fourier-Series coefficients numerically.
 2. Reconstruct a periodic complex signal from its coefficients.
 3. Manipulate coefficients using Fourier properties without recomputing them.
 4. Rank coefficients by energy and select subsets.
 5. Preserve original state across repeated experiments.
 6. Compute reconstruction MSE.
 7. Compute 1D CFT with `np.trapezoid`.
 8. Verify time shift, derivative, scaling, modulation, and Parseval.
 9. Compute 2D CFT through separable numerical integration.
 10. Design high-pass, low-pass, notch, and directional frequency masks.
 11. Reconstruct filtered images.
 12. Use magnitude, phase, wrapped phase error, thresholds, masks, sorting, broadcasting, indexing, and copies correctly.
-

2. Pattern Seen in the Previous-Year Questions

PYQ 1 - Differentiation Property

A signal such as

$$x(t) = 0.5 \cos(4t) + 0.5 \sin(6t)$$

was differentiated and the property

$$\mathcal{F} \left\{ \frac{dx}{dt} \right\} = j2\pi f X(f)$$

was verified numerically.

Students had to:

- compute the CFT of $x(t)$,
- derive dx/dt ,
- compute the CFT of the derivative,
- compare against $j2\pi f X(f)$,
- compare magnitude,
- compare phase,
- calculate MSE,
- repeat for second and third derivatives.

Reusable lesson

A property-verification question usually has two paths:

theoretical Fourier-domain prediction

versus

numerically measured transform

and then an error metric.

PYQ 2 - Gaussian Time Shift

Given

$$x(t) = e^{-t^2}$$

and

$$y(t) = x(t - t_0),$$

verify

$Y(f) = X(f)e^{-j2\pi ft_0}.$

Consequences:

$$|Y(f)| = |X(f)|$$

and

$$\angle Y(f) = \angle X(f) - 2\pi ft_0.$$

Reusable lesson

Time shift changes phase but not magnitude.

PYQ 3 - Frequency Shift + Time Scaling

A signal made from square and triangular components was modified by modulation and time compression.

Useful combined property:

$$x(at)e^{j2\pi f_0 t} \leftrightarrow \frac{1}{|a|} X\left(\frac{f - f_0}{a}\right).$$

Reusable lesson

The exam can combine two properties in one transformation.

PYQ 4 - Parseval

Students implemented a piecewise signal and verified

$$\int_{-\infty}^{\infty} |x(t)|^2 dt = \int_{-\infty}^{\infty} |X(f)|^2 df.$$

Reusable lesson

A theorem can be tested by numerically computing both sides.

PYQ 5 - Identify Sinusoidal Components

A complicated trigonometric expression was transformed, its spectral peaks were identified, and the original signal was reconstructed from the detected components.

Reusable lesson

Know how peaks in $|X(f)|$ correspond to frequency components.

PYQ 6 - Image Denoising

A noisy image had to be transformed, noise frequencies identified, those frequencies removed, and the image reconstructed enough to recognize a hidden letter.

Reusable lesson

The instructor may reuse Fourier machinery in a new application rather than asking the exact offline task.

3. Current Offline Assignment - Task 1

A 2D drawing is represented as one complex periodic signal:

$$f(t) = x(t) + jy(t)$$

over one period T .

Its complex Fourier series is

$$f(t) = \sum_{n=-\infty}^{\infty} c_n e^{jn\omega_0 t}$$

where

$$\omega_0 = \frac{2\pi}{T}$$

and

$$c_n = \frac{1}{T} \int_0^T f(t) e^{-jn\omega_0 t} dt.$$

With N harmonics on each side,

$$\hat{f}_N(t) = \sum_{n=-N}^N c_n e^{jn\omega_0 t}$$

and the number of terms is

$$2N + 1.$$

For $N = 150$,

$$2(150) + 1 = 301.$$

4. Core FourierEpicycles Code Pattern

```
class FourierEpicycles:
```

```
    def __init__(self, t, signal, n_harmonics):

        self.t = t
        self.signal = signal
        self.N = n_harmonics

        self.T = float(t[-1] - t[0])
        self.omega = 2 * np.pi / self.T

        self.coeffs = {}
```

Single coefficient:

```
def calculate_cn(self, n):

    kernel = np.exp(
        -1j * n * self.omega * self.t
    )

    return np.trapezoid(
        self.signal * kernel,
        self.t
    ) / self.T
```

All coefficients:

```
def calculate_all_coefficients(self):

    for n in range(-self.N, self.N + 1):

        self.coeffs[n] = \
            self.calculate_cn(n)
```

Reconstruction:

```
def approximate(self, t):

    result = 0
```

```

for n, c_n in self.coefs.items():

    result += (
        c_n
        * np.exp(
            1j * n * self.omega * t
        )
    )

return result

```

5. Meaning of an Epicycle Term

Each term

$$c_n e^{jn\omega_0 t}$$

is a rotating vector.

- radius:

$$|c_n|$$

- initial angle:

$$\angle c_n$$

- angular speed:

$$n\omega_0.$$

So:

- large $|c_n| \rightarrow$ large geometric contribution,
- large $|n| \rightarrow$ faster rotation and usually finer detail.

Do not confuse **coefficient magnitude** with **harmonic number**.

6. Fourier-Series Energy

For the finite coefficient set,

$$E_n = |c_n|^2$$

and

$$E_{\text{total}} = \sum_{n=-N}^N |c_n|^2.$$

This is the basis of the harmonic-pruning problem.

7. Given Problem - Energy-Preserving Harmonic Pruning

Goal

Implement:

```
prune_harmonics_by_energy(self, r)
```

where

$$r \in (0, 1].$$

Retain the **minimum number of most energetic harmonics** whose cumulative energy reaches at least

$$rE_{\text{total}}.$$

All other coefficients become zero.

Return:

- retained harmonic count,
- actual retained energy ratio.

Then implement reconstruction MSE:

$$\text{MSE} = \frac{1}{M} \sum_{i=1}^M |f(t_i) - \hat{f}(t_i)|^2.$$

Test:

$$r \in \{0.96, 0.98, 0.99, 1.00\}.$$

8. Energy-Pruning Solution

Step 1 - energies

```
energies = []  
  
for n in range(-self.N, self.N + 1):  
    energies.append(  
        np.abs(self.coefs[n]) ** 2  
    )
```

Step 2 - total and target

```
total = sum(energies)  
target = r * total
```

Step 3 - sort strongest first

```
sorted_indices = \
    np.argsort(energies)[::-1]
```

`np.argsort` gives the indices that would sort values in ascending order.

`[::-1]` reverses the order.

Step 4 - index to harmonic number

The list is ordered:

index:	0	1	...	N	...	2N
harmonic:	-N	-N+1	...	0	...	N

Therefore

$$n = \text{idx} - N.$$

Step 5 - accumulate

```
retained = []
actual = 0.0

for idx in sorted_indices:

    n = idx - self.N

    retained.append(n)

    actual += energies[idx]

    if actual >= target:
        break
```

Step 6 - zero discarded coefficients

```
for n in self.coeffs:

    if n not in retained:

        self.coeffs[n] = \
            0.0 + 0.0j
```

Step 7 - return

```
return (
    len(retained),
    actual / total
)
```

The actual ratio does not have to equal the target exactly.

Example:

12 harmonics -> 95.8%
13 harmonics -> 96.3%

For target 0.96, the correct actual ratio is 0.963.

9. Reconstruction Error

Correct method:

```
def evaluate_reconstruction_error(self):  
  
    reconstructed = \  
        self.approximate(self.t)  
  
    mse = np.mean(  
        np.abs(  
            self.signal  
            -  
            reconstructed  
        ) ** 2  
    )  
  
    return mse
```

For the complex drawing,

$$|f - \hat{f}|^2$$

is equivalent to

$$(\Delta x)^2 + (\Delta y)^2.$$

10. Critical Pitfall - Reusing a Pruned Object

Wrong:

```
for r in [0.96, 0.98, 0.99, 1.00]:
```

```
    fs.prune_harmonics_by_energy(r)
```

After the first pruning, many original coefficients are gone.

The next trial is no longer using the original spectrum.

Safe:

```
for r in ratios:
```

```
    fs = FourierEpicycles(  
        t,  
        z,  
        150  
    )
```

```
    fs.calculate_all_coefficients()
```

```
retained, actual = \
    fs.prune_harmonics_by_energy(r)
```

Alternative:

```
original_coeffs = self.coeffs.copy()
```

and restore it before each experiment.

11. Expected Pruning Trend

As

$$r \uparrow,$$

generally

harmonics retained $\uparrow$

and

reconstruction MSE $\downarrow$.

For $N = 150$ and $r = 1$, expect all 301 harmonics to remain.

12. Fourier-Series Properties to Memorize

Time shift

If

$$g(t) = f(t - t_0),$$

then

$$d_n = c_n e^{-jn\omega_0 t_0}.$$

Therefore

$$|d_n| = |c_n|.$$

Time reversal

$$f(-t) \Rightarrow d_n = c_{-n}$$

Complex scaling / rotation

If

$$g(t) = Af(t),$$

then

$$d_n = Ac_n.$$

If

$$A = \rho e^{j\theta},$$

then ρ scales the drawing and $e^{j\theta}$ rotates it.

Spatial translation of the drawing

If

$$g(t) = f(t) + B,$$

then only the DC coefficient changes:

$$d_0 = c_0 + B$$

and for $n \neq 0$,

$$d_n = c_n.$$

Differentiation

$$d_n = jn\omega_0 c_n$$

because

$$\frac{d}{dt} e^{jn\omega_0 t} = jn\omega_0 e^{jn\omega_0 t}.$$

Magnitude:

$$|d_n| = |n|\omega_0|c_n|.$$

High-order harmonics are amplified more strongly.

Fourier-Series Parseval

$$\frac{1}{T} \int_0^T |f(t)|^2 dt = \sum_n |c_n|^2.$$

For finite N , compare the numerical time-domain energy with the finite coefficient sum.

13. Warm-Up Problem 1 - Top- K Harmonics

Keep exactly the K largest-energy harmonics.

Implement:

```
keep_top_harmonics(self, k)
```

Return

$$R_E = \frac{\sum_{\text{retained}} |c_n|^2}{\sum_n |c_n|^2}.$$

Test:

```
K = [10, 20, 50, 100, 301]
```

Expected:

$$K \uparrow \Rightarrow R_E \uparrow$$

and

$$K \uparrow \Rightarrow \text{MSE} \downarrow.$$

14. Warm-Up Problem 2 - Fourier-Series Time Shift

Given

$$g(t) = f(t - t_0),$$

predict:

$$d_n = c_n e^{-jn\omega_0 t_0}.$$

Implement:

```
predict_shifted_coefficients(self, t0)
```

and reconstruct using the predicted coefficients.

Verify:

$$|d_n| \approx |c_n|.$$

Main trap:

$f(t - t_0)$ changes the starting point along the closed path; it does not physically translate the shape.

15. Warm-Up Problem 3 - Velocity Signal

Use

$$d_n = jn\omega_0 c_n$$

to reconstruct

$$f'(t).$$

Implement:

```
derivative_coefficients(self)
```

and

```
approximate_derivative(self, t)
```

Use

```
np.gradient(signal, t)
```

only as a numerical verification if allowed.

16. Warm-Up Problem 4 - Low-Pass Harmonic Reconstruction

Keep only

$$|n| \leq K.$$

Thus

$$c'_n = \begin{cases} c_n, & |n| \leq K \\ 0, & |n| > K. \end{cases}$$

The retained harmonic count is

$$2K + 1.$$

Test:

```
cutoffs = [3, 5, 10, 20, 50, 150]
```

Compare with energy pruning.

For the same number of coefficients, energy pruning is usually more efficient because it selects by actual contribution.

17. Warm-Up Problem 5 - Coefficient Quantization

For

$$c_n = a_n + jb_n,$$

quantize with

$$Q(x) = \Delta \operatorname{round}\left(\frac{x}{\Delta}\right)$$

and

$$\tilde{c}_n = Q(a_n) + jQ(b_n).$$

Example:

$$0.137 + 0.064j$$

with

$$\Delta = 0.05$$

becomes approximately

$$0.15 + 0.05j.$$

Measure:

- coefficient MSE,
- reconstruction MSE.

18. Hard Problem 1 - Symmetric Energy-Constrained Compression

Require $+n$ and $-n$ to be retained together.

Pair energy:

$$E_{\text{pair}}(n) = |c_n|^2 + |c_{-n}|^2.$$

Always retain c_0 .

Implement:

```
prune_symmetric_by_energy(self, r)
```

Return:

- nonzero coefficient count,
- retained pair count,
- actual energy ratio.

Then compare against ordinary energy pruning for

```
ratios = [0.90, 0.95, 0.98, 0.99]
```

using MSE.

Solution idea

```
pair_energies = []

for n in range(1, self.N + 1):

    e = (
        np.abs(self.coeffs[n]) ** 2
        +
        np.abs(self.coeffs[-n]) ** 2
    )

    pair_energies.append(e)
```

Sort:

```
order = \
    np.argsort(pair_energies)[::-1]
```

Because list index 0 corresponds to pair $n = 1$,

```
n = idx + 1
```

Start with:

```
retained = {0}
```

```
actual = \
    np.abs(self.coeffs[0]) ** 2
```

Then add both $+n$ and $-n$.

Symmetric pruning may retain more coefficients because it can be forced to keep a weak partner.

19. Hard Problem 2 - Minimum Harmonics for an MSE Constraint

Given a tolerance ϵ , keep the smallest number of harmonics such that

$$\boxed{\text{MSE} \leq \epsilon}.$$

Add harmonics in descending energy order.

Implement:

```
rank_harmonics_by_energy(self)
and
prune_for_mse(self, tolerance)
```

Test:

```
tolerances = [
    1e-2,
    5e-3,
    1e-3,
    5e-4
]
```

Solution pattern

```
harmonics = []
energies = []

for n in range(-self.N, self.N + 1):

    harmonics.append(n)

    energies.append(
        np.abs(self.coefs[n]) ** 2
    )
```

```
order = np.argsort(energies)[::-1]
```

Keep original:

```
original = self.coefs.copy()
```

Create zeros:

```
trial = {}
```

```
for n in original:
    trial[n] = 0.0 + 0.0j
```

Add one at a time and reconstruct.

Stop at the **first** MSE satisfying the tolerance.

20. Hard Problem 3 - Combined Time Shift + Rotation + Scale + Translation

Define

$$g(t) = Ae^{j\theta}f(t - t_0) + B.$$

Interpretation:

- t_0 : parameter/time shift,
- A : spatial scaling,
- $e^{j\theta}$: spatial rotation,
- B : geometric translation.

Start from

$$f(t - t_0) = \sum_n c_n e^{jn\omega_0(t-t_0)}$$

so

$$f(t - t_0) = \sum_n c_n e^{-jn\omega_0 t_0} e^{jn\omega_0 t}.$$

Therefore:

$$d_n = Ae^{j\theta}c_n e^{-jn\omega_0 t_0}, \quad n \neq 0$$

and because B is a constant/DC term,

$$d_0 = Ae^{j\theta}c_0 + B.$$

Implementation:

```
def transform_coefficients(
    self,
    A,
    theta,
    t0,
    B
):
    transformed = {}

    for n, c_n in self.coeffs.items():
        d_n = (
            A
            * np.exp(1j * theta)
            * c_n
            * np.exp(
                -1j
                * n
                * self.omega
                * t0
            )
        )

        if n == 0:
            d_n += B

        transformed[n] = d_n

    return transformed
```

Verification

Use two paths.

Theoretical:

$$c_n \rightarrow d_n^{predicted}.$$

Numerical:

1. construct $g(t)$,
2. create another Fourier object,
3. calculate $d_n^{measured}$ using integration.

Then compare magnitude and phase.

Magnitude MSE:

$$\text{MSE}_{mag} = \frac{1}{2N+1} \sum_n (|d_n^{measured}| - |d_n^{predicted}|)^2.$$

Phase must be wrapped:

$$\Delta\phi = \angle \left[e^{j(\phi_m - \phi_p)} \right].$$

21. Hard Problem 4 - Harmonic-Band Detail Enhancement

Split into:

$$L : |n| \leq 10,$$

$$M : 10 < |n| \leq 50,$$

$$H : |n| > 50.$$

Apply

$$d_n = \begin{cases} c_n, & |n| \leq 10 \\ \alpha c_n, & 10 < |n| \leq 50 \\ \beta c_n, & |n| > 50. \end{cases}$$

Take

$$\alpha = 1.2$$

and search

```
beta_candidates = [  
    1.0, 1.25, 1.5, 1.75,  
    2.0, 2.5, 3.0, 4.0  
]
```

for the largest β satisfying

$$\text{MSE} \leq 0.002.$$

Also compute low/mid/high energy ratios.

Important:

Every candidate β must start from the original coefficients.

22. Hard Problem 5 - Joint Pruning + Quantization

Pipeline:

$$\text{original} \rightarrow \text{energy pruning} \rightarrow \text{quantization} \rightarrow \text{reconstruction}.$$

Test:

```
energy_ratios = [
    0.90,
    0.95,
    0.98,
    0.99
]
```

```
deltas = [
    0.001,
    0.005,
    0.01,
    0.02
]
```

Total configurations:

$$4 \times 4 = 16.$$

For each calculate:

- retained harmonics,
- actual energy ratio,
- coefficient MSE,
- signal MSE.

Coefficient MSE:

$$\text{MSE}_c = \frac{1}{2N+1} \sum_n |c_n - \tilde{c}_n|^2.$$

Signal MSE:

$$\text{MSE}_{\text{signal}} = \frac{1}{M} \sum_i |f(t_i) - \hat{f}(t_i)|^2.$$

A configuration is valid if

$$\text{MSE}_{\text{signal}} \leq 0.001.$$

Best configuration:

1. smallest retained harmonic count,
2. if tied, largest Δ .

This is an **optimization** problem, not a “find the lowest MSE” problem.

23. Additional Mock Variants Based on This Year’s Assignment

These are other plausible exam variants discussed during preparation.

Variant A - Parameter Shift vs Geometric Translation

Compare:

$$f(t - t_0)$$

with

$$f(t) + B.$$

The first changes the starting point along the same closed curve.

The second moves the whole curve in the complex plane.

Coefficient rules:

$$f(t - t_0) \Rightarrow c_n e^{-jn\omega_0 t_0}$$

and

$$f(t) + B \Rightarrow \begin{cases} c_0 + B, & n = 0 \\ c_n, & n \neq 0. \end{cases}$$

Variant B - Time Reversal

Given

$$g(t) = f(-t),$$

verify

$$\boxed{d_n = c_{-n}}.$$

Numerically:

1. construct the reversed periodic signal,
 2. calculate its coefficients,
 3. compare against the index-reversed original coefficient set.
-

Variant C - Fourier-Series Parseval

Compute:

$$E_t = \frac{1}{T} \int_0^T |f(t)|^2 dt$$

and

$$E_c = \sum_{n=-N}^N |c_n|^2.$$

Then measure:

`error = abs(E_t - E_c)`

or relative error.

Variant D - Harmonic Smoothing

Keep only low-order harmonics.

Observe how reducing high-order terms removes sharp geometric details.

Possible task:

Find the smallest cutoff K such that MSE is below a specified tolerance.

This combines low-pass filtering with search.

Variant E - 2D Low-Pass Blurring

Reverse the supplied high-pass edge detector.

Instead of deleting frequencies near the center, keep only the central region and remove frequencies outside a radius.

Expected:

- small cutoff $\rightarrow$ strong blur,
 - larger cutoff $\rightarrow$ more detail.
-

Variant F - 2D Spatial Shift

For

$$g(x, y) = I(x - x_0, y - y_0),$$

verify

$$G(u, v) = F(u, v)e^{-j2\pi(ux_0+vy_0)}.$$

Magnitude is unchanged.

Variant G - Conjugate Symmetry

For a real image,

$$F(-u, -v) = F^*(u, v).$$

Possible task:

- compare symmetric frequency locations,
 - calculate MSE of real and imaginary symmetry conditions.
-

Variant H - Periodic Stripe-Noise Removal

Periodic image noise creates concentrated off-center peaks.

Steps:

1. compute 2D CFT,
 2. inspect log magnitude,
 3. identify noise peaks,
 4. zero small neighborhoods around them,
 5. also remove the conjugate/symmetric peaks,
 6. inverse-transform,
 7. compare the result.
-

Variant I - Directional Edge Extraction

Build a mask that keeps frequencies mainly along one frequency axis.

Possible objective:

- emphasize vertical edges,
- emphasize horizontal edges.

This tests understanding of u, v , not just radial filtering.

Variant J - 2D Cosine Modulation

Multiply the image by a cosine pattern.

Using

$$\cos(2\pi u_0 x) = \frac{1}{2}e^{j2\pi u_0 x} + \frac{1}{2}e^{-j2\pi u_0 x},$$

the spectrum should split/shift into two copies.

A plausible online could ask you to generate the modulated image and verify the shifted spectra.

Variant K - Anisotropic Spatial Scaling

If

$$g(x, y) = I(ax, by),$$

then the spectrum scales differently along u and v .

This is harder because horizontal and vertical scaling must be handled independently.

Variant L - 2D Parseval

Verify:

$$\iint |I(x, y)|^2 dx dy = \iint |F(u, v)|^2 du dv$$

using nested `np.trapezoid`.

24. Continuous Fourier Transform Fundamentals

Using ordinary frequency f :

$$X(f) = \int_{-\infty}^{\infty} x(t)e^{-j2\pi ft} dt$$

and inverse:

$$x(t) = \int_{-\infty}^{\infty} X(f)e^{j2\pi ft} df.$$

Using angular frequency ω :

$$X(\omega) = \int_{-\infty}^{\infty} x(t)e^{-j\omega t} dt$$

and

$$x(t) = \frac{1}{2\pi} \int_{-\infty}^{\infty} X(\omega)e^{j\omega t} d\omega.$$

25. When to Use 2π

If the variable is ordinary frequency f in cycles per second:

$$e^{-j2\pi ft}.$$

If the variable is angular frequency ω in radians per second:

$$e^{-j\omega t}.$$

Because:

$$\omega = 2\pi f.$$

Do not mix the two conventions.

26. Numerical 1D CFT Template

```
class CFTAnalyzer:
    def __init__(self, t, signal):
        self.t = t
        self.signal = signal
```

```
def compute(self, frequencies):

    ft = (
        frequencies[:, None]
        * self.t[None, :]
    )

    kernel = np.exp(
        -1j
        * 2
        * np.pi
        * ft
    )

    return np.trapezoid(
        self.signal[None, :]
        * kernel,
        self.t,
        axis=-1
    )
```

Shape reasoning:

```
frequencies[:, None] -> (F, 1)
t[None, :]           -> (1, T)
product              -> (F, T)
```

Each row corresponds to one candidate frequency.

Then:

```
axis=-1
```

integrates across time.

27. CFT Properties to Memorize

Time shift

$$x(t - t_0) \leftrightarrow X(f)e^{-j2\pi f t_0}$$

Frequency shift / modulation

$$x(t)e^{j2\pi f_0 t} \leftrightarrow X(f - f_0)$$

Time scaling

$$x(at) \leftrightarrow \frac{1}{|a|} X\left(\frac{f}{a}\right)$$

Time reversal

$$x(-t) \leftrightarrow X(-f)$$

Differentiation

$$\frac{dx}{dt} \leftrightarrow j2\pi f X(f)$$

Repeated derivative:

$$\frac{d^k x}{dt^k} \leftrightarrow (j2\pi f)^k X(f)$$

Multiplication / convolution

$$x(t)y(t) \leftrightarrow X(f) * Y(f)$$

and

$$x(t) * y(t) \leftrightarrow X(f)Y(f)$$

Parseval

For the f -based convention:

$$\int |x(t)|^2 dt = \int |X(f)|^2 df.$$

28. Why Time Scaling Gives $1/|a|$

Let:

$$y(t) = x(at).$$

Then:

$$Y(f) = \int x(at)e^{-j2\pi ft} dt.$$

Use:

$$\tau = at.$$

Then:

$$t = \frac{\tau}{a}.$$

Accounting for the direction of integration when $a < 0$:

$$dt = \frac{d\tau}{|a|}.$$

Therefore:

$$Y(f) = \frac{1}{|a|} \int x(\tau) e^{-j2\pi(f/a)\tau} d\tau.$$

Hence:

$$Y(f) = \frac{1}{|a|} X\left(\frac{f}{a}\right).$$

29. Magnitude and Phase

For

$$X = a + jb,$$

magnitude:

$$|X| = \sqrt{a^2 + b^2}$$

Python:

```
magnitude = np.abs(X)
```

Phase:

```
phase = np.angle(X)
```

returns radians.

30. Why Phase Near Zero Magnitude Is Unreliable

When:

$$|X| \approx 0,$$

phase becomes numerically unstable and physically unimportant.

Use a relative threshold:

```
threshold = (
    1e-6
    * max(
        np.max(np.abs(X)),
        np.max(np.abs(Y))
    )
)
```

Then:

```
mask = (
    (np.abs(X) > threshold)
    &
    (np.abs(Y) > threshold)
)
```

A relative threshold adapts to signal scale better than a fixed absolute threshold.

31. Wrapped Phase Error

Plain phase subtraction is dangerous because phase is circular.

Example:

$$179^\circ$$

and

$$-179^\circ$$

are only 2° apart.

Use:

$$\Delta\phi = \angle \left(e^{j(\phi_m - \phi_p)} \right).$$

Python:

```
phase_error = np.angle(
    np.exp(
        1j * (
            measured_phase
            -
            predicted_phase
        )
    )
)
```

Then:

```
phase_mse = np.mean(
    phase_error[mask] ** 2
)
```

32. Current Offline Assignment - 2D CFT

For grayscale image $I(x, y)$:

$$F(u, v) = \iint I(x, y) e^{-j2\pi(ux+vy)} dx dy$$

where:

- x : horizontal position,
- y : vertical position,
- u : horizontal spatial frequency,
- v : vertical spatial frequency.

Inverse:

$$I(x, y) = \iint F(u, v) e^{j2\pi(ux+vy)} du dv.$$

33. Image Array Orientation

NumPy image shape:

```
image.shape == (rows, columns)
```

Conceptually:

```
row    -> y  
column -> x
```

Therefore:

```
I[y_index, x_index]
```

not:

```
I[x_index, y_index]
```

If:

```
height, width = I.shape
```

then:

```
x = np.linspace(-1, 1, width)  
y = np.linspace(-1, 1, height)
```

34. Spatial Frequency Intuition

Spatial frequency means:

How quickly image intensity changes with position.

Smooth:

```
100 101 101 102 102
```

→ slow variation → low frequency.

Edge:

```
20 20 20 230 230
```

→ sudden variation → high frequency.

Therefore:

- low frequency → background / smooth regions,
- high frequency → edges / fine texture.

Brightness itself is not frequency.

35. Real and Imaginary Parts of the 2D CFT

Using:

$$e^{-j\theta} = \cos \theta - j \sin \theta,$$

we get:

$$\Re\{F(u, v)\} = \iint I(x, y) \cos(2\pi(ux + vy)) dx dy$$

and

$$\Im\{F(u, v)\} = - \iint I(x, y) \sin(2\pi(ux + vy)) dx dy .$$

The current assignment explicitly uses real cosine/sine arithmetic.

36. Separability

Direct computation over all

$$x, y, u, v$$

would be roughly $O(N^4)$.

Instead:

1. integrate over x for every row and u ,
2. integrate the intermediate result over y for every u, v .

This is the essential optimization.

Angle-sum identities:

$$\cos(A + B) = \cos A \cos B - \sin A \sin B$$

$$\sin(A + B) = \sin A \cos B + \cos A \sin B.$$

These let the ux and vy dependence separate.

37. Loop-Based 2D CFT Pattern

First stage:

```
a = np.zeros(
    (len(self.y), len(self.u))
)

b = np.zeros(
    (len(self.y), len(self.u))
)

for u_idx, u_val in enumerate(self.u):

    cos_part = np.cos(
        2 * np.pi
        * u_val
        * self.x
    )

    sin_part = np.sin(
        2 * np.pi
        * u_val
```

```

        * self.x
    )

    for y_idx in range(len(self.y)):

        row = self.I[y_idx, :]

        a[y_idx, u_idx] = \
            np.trapezoid(
                row * cos_part,
                self.x
            )

        b[y_idx, u_idx] = \
            -np.trapezoid(
                row * sin_part,
                self.x
            )

```

Second stage:

```

real = np.zeros(
    (len(self.v), len(self.u))
)

imag = np.zeros(
    (len(self.v), len(self.u))
)

for v_idx, v_val in enumerate(self.v):

    cos_part = np.cos(
        2 * np.pi
        * v_val
        * self.y
    )

    sin_part = np.sin(
        2 * np.pi
        * v_val
        * self.y
    )

    for u_idx in range(len(self.u)):

        a_col = a[:, u_idx]
        b_col = b[:, u_idx]

        real_integrand = (
            a_col * cos_part
            +
            b_col * sin_part
        )

        imag_integrand = (
            b_col * cos_part
            -
            a_col * sin_part

```

```

    )

    real[v_idx, u_idx] = \
        np.trapezoid(
            real_integrand,
            self.y
        )

    imag[v_idx, u_idx] = \
        np.trapezoid(
            imag_integrand,
            self.y
        )

```

38. Magnitude Spectrum

$$|F(u, v)| = \sqrt{\Re(F)^2 + \Im(F)^2}$$

Python:

```

magnitude = np.hypot(
    real,
    imag
)

```

or:

```

magnitude = np.sqrt(
    real**2 + imag**2
)

```

For display:

```

plt.imshow(
    np.log1p(magnitude),
    cmap="gray",
    origin="lower"
)

```

`np.log1p(x)` means:

$$\log(1 + x).$$

It makes weaker spectral components visible.

39. High-Pass Filtering

For index-based radial filtering:

```

rows, cols = real.shape

```

```

cy = rows // 2
cx = cols // 2

```

Then:

```

for i in range(rows):
    for j in range(cols):
        radius = np.sqrt(
            (i - cy) ** 2
            +
            (j - cx) ** 2
        )

        if radius <= cutoff:

            real[i, j] = 0
            imag[i, j] = 0

```

This removes the low-frequency center and preserves edge-heavy content.

40. Low-Pass Filtering

Reverse the condition:

```

if radius > cutoff:

    real[i, j] = 0
    imag[i, j] = 0

```

This keeps the center and suppresses high frequencies.

Expected: smoothing / blur.

41. Notch Filtering

If noise appears at isolated spectral peaks, remove only neighborhoods around those peaks.

General mask idea:

```

distance = np.sqrt(
    (U - u0) ** 2
    +
    (V - v0) ** 2
)

mask = distance <= radius

real[mask] = 0
imag[mask] = 0

```

For real images, usually also remove the symmetric/conjugate peak.

42. Directional Filtering

Create:

```

U, V = np.meshgrid(
    self.u,

```

```

    self.v
)

```

Then conditions like:

```
np.abs(U) > cutoff
```

or

```
np.abs(V) > cutoff
```

can isolate frequency regions by direction.

Be careful:

The orientation of a frequency component and the orientation of the visible edge are related but not identical in the naïve “same direction” sense. Think in terms of the direction in which intensity changes.

43. Inverse 2D CFT

$$I(x, y) = \iint F(u, v) e^{j2\pi(ux+vy)} du dv$$

Again separate cosine and sine and perform two 1D integrations.

After high-pass filtering, the reconstructed signal can contain positive and negative values because the average/DC brightness was removed.

Typical edge display:

```
edge_map = np.abs(edges)
```

```
if edge_map.max() > 0:
```

```
    edge_map = \
        edge_map / edge_map.max()
```

```
edge_map = 1 - edge_map
```

44. 2D Parseval Numerical Pattern

Spatial energy:

```
temp = np.trapezoid(
    np.abs(I) ** 2,
    x,
    axis=-1
)
```

```
E_space = np.trapezoid(
    temp,
    y
)
```

Frequency energy:

```
mag2 = (
    real**2
    +

```

```

    imag**2
)

temp = np.trapezoid(
    mag2,
    u,
    axis=-1
)

E_freq = np.trapezoid(
    temp,
    v
)

Compare:
error = abs(
    E_space - E_freq
)

or relative error.

```

45. NumPy / Python Functions You Need

np.linspace

```

t = np.linspace(
    -5,
    5,
    2000
)

```

Creates evenly spaced samples including endpoints by default.

np.arange

```

n = np.arange(
    -N,
    N + 1
)

```

Upper endpoint is excluded, so use $N + 1$.

np.abs

For complex:

```
np.abs(c_n)
```

returns magnitude.

Energy:

```
np.abs(c_n) ** 2
```

np.angle

`phase = np.angle(X)`

returns angle in radians.

np.exp

`np.exp(1j * theta)`

represents

$$e^{j\theta}.$$

Used in:

- Fourier kernels,
 - rotation,
 - phase shift,
 - phase wrapping.
-

np.trapezoid

1D:

```
value = np.trapezoid(  
    y,  
    x  
)
```

Along an array axis:

```
value = np.trapezoid(  
    F,  
    x,  
    axis=-1  
)
```

46. Understanding axis

Suppose:

`F.shape == (V, U)`

and columns correspond to u .

Then:

```
np.trapezoid(  
    F,  
    u,  
    axis=-1  
)
```

integrates along columns and returns shape:

`(V,)`

Always write the shape before choosing `axis`.

47. np.argsort

Example:

```
values = np.array(
    [4, 10, 2, 8]
)
```

Ascending index order:

```
idx = np.argsort(values)
```

Descending:

```
idx = np.argsort(values)[::-1]
```

This is the key function for:

- energy pruning,
 - top- K ,
 - rank-based searches.
-

48. Dictionary Sorting

Given:

```
temp = {}
```

```
for n, c_n in self.coefs.items():
    temp[n] = \
        np.abs(c_n) ** 2
```

You can sort values using:

```
sorted_items = sorted(
    temp.items(),
    key=lambda item: item[1],
    reverse=True
)
```

Important:

```
sorted(temp)
```

sorts only dictionary **keys**.

If you are uncomfortable with `lambda`, use arrays and `np.argsort` in an exam.

49. [::-1]

```
array[::-1]
```

reverses an array/list.

Typical use:

```
np.argsort(values)[::-1]
```

for descending order.

But:

Reversing sample order is not always equivalent to evaluating $x(-t)$. It depends on the sampling grid.

50. Boolean Masks

A mask contains `True/False`.

Example:

```
mask = magnitude > threshold
```

Then:

```
phase_error[mask]
```

keeps only valid entries.

Combine conditions:

```
mask = (
    (np.abs(X) > threshold)
    &
    (np.abs(Y) > threshold)
)
```

Use:

- `&` → elementwise AND,
- `|` → elementwise OR,
- `~` → elementwise NOT.

Do not use Python `and` / `or` with NumPy arrays.

51. 2D Masks

```
U, V = np.meshgrid(
    u,
    v
)
```

```
radius = np.sqrt(
    U**2 + V**2
)
```

```
mask = radius < cutoff
```

Then:

```
real[mask] = 0
imag[mask] = 0
```

52. `np.meshgrid`

If:

```
u.shape == (U,)
v.shape == (V,)

then:

U_grid, V_grid = \
    np.meshgrid(u, v)

creates arrays of shape:

(V, U)

with:

U_grid[v_idx, u_idx] = u[u_idx]
V_grid[v_idx, u_idx] = v[v_idx]
```

53. Broadcasting with None

Suppose:

```
frequencies.shape == (F,)
t.shape == (T,)
```

Then:

```
frequencies[:, None]
```

shape:

```
(F, 1)
```

and:

```
t[None, :]
```

shape:

```
(1, T)
```

Multiplication broadcasts to:

```
(F, T)
```

Example:

```
ft = (
    frequencies[:, None]
    * t[None, :]
)
```

54. 3D Broadcasting for Images

If:

```
I.shape == (Y, X)
```

then:

```
I[None, :, :]
```

has shape:

```
(1, Y, X)
```

If:

```
cos_ux.shape == (U, X)
```

then:

```
cos_ux[:, None, :]
```

has shape:

```
(U, 1, X)
```

Multiplication broadcasts to:

```
(U, Y, X)
```

meaning:

every candidate u , every image row y , every spatial sample x .

55. np.mean

Signal MSE:

```
mse = np.mean(  
    np.abs(  
        actual - predicted  
    ) ** 2  
)
```

This directly represents the average squared magnitude error.

56. np.gradient

Numerical derivative:

```
dx_dt = np.gradient(  
    signal,  
    t  
)
```

Use it for verification if permitted.

Do not replace a required Fourier-domain derivation with `np.gradient`.

57. .copy()

Very important.

Dictionary:

```
original = \  
    self.coeffs.copy()
```

Array:

```
real_f = real.copy()
```

Use copies before:

- pruning,
- quantizing,
- filtering,

- testing candidate parameters.
-

58. enumerate

```
for idx, value in enumerate(self.u):
```

gives both index and value.

Very useful in transform loops.

59. Complex Parts

```
c.real
```

```
c.imag
```

Rebuild:

```
c_new = \
    real_part \
    + 1j * imag_part
```

60. np.hypot

```
magnitude = np.hypot(
    real,
    imag
)
```

Equivalent to:

```
np.sqrt(
    real**2 + imag**2
)
```

61. np.log1p

```
np.log1p(magnitude)
```

means:

$$\log(1 + \text{magnitude}).$$

Useful for spectrum visualization.

62. Sets and Membership

For selected harmonics:

```
retained = set()
```

```
retained.add(n)
```

Then:

```
if n not in retained:
```

Membership in a set is convenient and efficient.

For very small problems, a list is also acceptable.

63. Reusable Error Metrics

Signal MSE

```
mse = np.mean(
    np.abs(
        true_signal
        -
        predicted_signal
    ) ** 2
)
```

Magnitude MSE

```
mse_mag = np.mean(
    (
        np.abs(measured)
        -
        np.abs(predicted)
    ) ** 2
)
```

Wrapped phase MSE

```
phase_diff = np.angle(
    np.exp(
        1j
        * (
            np.angle(measured)
            -
            np.angle(predicted)
        )
    )
)

mse_phase = np.mean(
    phase_diff[mask] ** 2
)
```

Relative energy error

```
relative_error = (
    np.abs(E1 - E2)
    / np.abs(E1)
)
```

64. Search / Optimization Loop Pattern

Many difficult exam questions say:

Try several candidates and choose the best valid one.

General structure:

```
best = None

for candidate in candidates:

    # start from fresh state

    # apply candidate

    # calculate error

    if error <= tolerance:

        if best is None:
            best = result

        elif candidate_is_better:
            best = result
```

Use straightforward loops instead of clever one-liners during an exam.

65. Common Pitfalls

Pitfall 1 - Mutating the original coefficient set

If you do:

```
self.coeffs[n] = 0
```

the next experiment sees the modified spectrum.

Use a fresh object or `.copy()`.

Pitfall 2 - Forgetting negative harmonics

Correct:

```
range(-N, N + 1)
```

Pitfall 3 - Forgetting $1/T$

Analysis:

$$c_n = \frac{1}{T} \int f(t) e^{-jn\omega_0 t} dt.$$

Reconstruction does not get another $1/T$.

Pitfall 4 - Wrong exponent sign

Analysis:

$$e^{-jn\omega_0 t}$$

Synthesis:

$$e^{+jn\omega_0 t}.$$

CFT:

$$e^{-j2\pi ft}$$

Inverse CFT:

$$e^{+j2\pi ft}.$$

Pitfall 5 - Time shift vs drawing translation

$$f(t - t_0)$$

changes the tracing start.

$$f(t) + B$$

moves the entire shape.

Pitfall 6 - Phase comparison near zero magnitude

Use a threshold mask.

Pitfall 7 - Plain phase subtraction

Use wrapped phase error.

Pitfall 8 - Wrong image indexing

Remember:

`I[y, x]`

Pitfall 9 - Wrong integration axis

Always inspect shape before `np.trapezoid`.

Pitfall 10 - Using spatial axes as frequency axes

In the 2D CFT use:

```
self.u  
self.v
```

not:

```
self.x  
self.y
```

as the frequency variables.

Pitfall 11 - Direct $O(N^4)$ transform

Use separability.

Pitfall 12 - FFT when prohibited

Do not use:

```
np.fft  
scipy.fft
```

if the problem prohibits them.

Pitfall 13 - Re-quantizing or re-amplifying already modified coefficients

Every trial must start from the same reference spectrum.

Pitfall 14 - Losing harmonic identity after sorting

If the energy list corresponds to:

$$-N, \dots, N,$$

then:

$$\boxed{n = idx - N}.$$

Pitfall 15 - Assuming actual energy ratio equals target

Whole harmonics are discrete choices, so actual retained energy usually slightly exceeds the threshold.

66. How to Classify a New Exam Question

Type A - Property verification

Examples:

- time shift,

- derivative,
- scaling,
- reversal,
- modulation.

Ask:

1. What is the theoretical relationship?
 2. What stays unchanged?
 3. What is measured numerically?
 4. Do I need magnitude, phase, or signal MSE?
-

Type B - Coefficient processing / compression

Examples:

- pruning,
- top- K ,
- MSE-based selection,
- pairwise pruning,
- quantization.

Ask:

1. What metric ranks coefficients?
 2. What is the stopping condition?
 3. Does the method mutate state?
 4. What must be preserved between trials?
 5. What is the final error metric?
-

Type C - Image frequency filtering

Ask:

1. Which frequency region should be removed?
 2. Is the mask radial, notch, or directional?
 3. Are coordinates index-based or physical u, v ?
 4. Must conjugate symmetry be preserved?
 5. What should reconstruction look like?
-

Type D - Optimization/search

Ask:

1. What combinations must be tried?
 2. What condition makes a result valid?
 3. What quantity is optimized?
 4. What is the tie-breaker?
 5. Does every trial start from fresh state?
-

67. 30-40 Minute Exam Strategy

First 3-5 minutes

Write the mathematical rule first.

Examples:

$$E_n = |c_n|^2$$

or

$$d_n = Ae^{j\theta}c_n e^{-jn\omega_0 t_0}.$$

This prevents implementing the wrong operation.

Next 15-20 minutes

Implement only the new required methods.

Do not rewrite provided framework code.

Use explicit loops if they are safer.

Next 5-10 minutes

Implement the required experiments in `main`.

Make sure each parameter setting starts from fresh state.

Final 5 minutes

Check:

- exponent signs,
 - harmonic range,
 - $N + 1$,
 - `axis`,
 - `.copy()`,
 - phase threshold,
 - output filenames,
 - print format,
 - no prohibited FFT.
-

68. Minimal Templates to Memorize

Fourier-Series coefficient

```
c_n = np.trapezoid(
    signal
    * np.exp(
        -1j
        * n
        * omega
        * t
    ),
    t
) / T
```

Fourier-Series reconstruction

```
result = 0

for n, c_n in coeffs.items():

    result += (
        c_n
        * np.exp(
            1j
            * n
            * omega
            * t
        )
    )
```

Energy

```
energy = np.abs(c_n) ** 2
```

Descending sort

```
order = \
    np.argsort(values)[::-1]
```

MSE

```
mse = np.mean(
    np.abs(
        actual - predicted
    ) ** 2
)
```

Phase wrap

```
diff = np.angle(
    np.exp(
        1j
        * (
            measured
            -
            predicted
        )
    )
)
```

Magnitude mask

```
threshold = (
    1e-6
    * max(
```

```

        np.max(np.abs(A)),
        np.max(np.abs(B))
    )
)

mask = (
    (np.abs(A) > threshold)
    &
    (np.abs(B) > threshold)
)

```

Fresh-object experiment

```

for parameter in parameters:

    fs = FourierEpicycles(
        t,
        signal,
        N
    )

    fs.calculate_all_coefficients()

# perform this trial only

```

69. Final Formula Sheet

Fourier Series

$$\omega_0 = \frac{2\pi}{T}$$

$$c_n = \frac{1}{T} \int_0^T f(t) e^{-jn\omega_0 t} dt$$

$$f(t) = \sum_n c_n e^{jn\omega_0 t}$$

$$E_n = |c_n|^2$$

$$E_{\text{total}} = \sum_n |c_n|^2$$

Time shift:

$$f(t - t_0) \Rightarrow c_n e^{-jn\omega_0 t_0}$$

Time reversal:

$$f(-t) \Rightarrow c_{-n}$$

Derivative:

$$f'(t) \Rightarrow jn\omega_0 c_n$$

Scale/rotation:

$$Af(t) \Rightarrow Ac_n$$

Spatial translation:

$$f(t) + B \Rightarrow c_0 + B$$

for the DC term only.

CFT using f

$$X(f) = \int x(t)e^{-j2\pi ft} dt$$

$$x(t) = \int X(f)e^{j2\pi ft} df$$

Time shift:

$$x(t - t_0) \leftrightarrow X(f)e^{-j2\pi ft_0}$$

Scaling:

$$x(at) \leftrightarrow \frac{1}{|a|} X(f/a)$$

Modulation:

$$x(t)e^{j2\pi f_0 t} \leftrightarrow X(f - f_0)$$

Derivative:

$$x'(t) \leftrightarrow j2\pi f X(f)$$

Parseval:

$$\int |x(t)|^2 dt = \int |X(f)|^2 df$$

2D CFT

$$F(u, v) = \iint I(x, y) e^{-j2\pi(ux+vy)} dx dy$$

$$I(x, y) = \iint F(u, v) e^{j2\pi(ux+vy)} du dv$$

Spatial shift:

$$I(x - x_0, y - y_0) \leftrightarrow F(u, v) e^{-j2\pi(ux_0+vy_0)}$$

Magnitude:

$$|F| = \sqrt{\Re(F)^2 + \Im(F)^2}$$

For real images:

$$F(-u, -v) = F^*(u, v)$$

70. Preparation Priority

If time is limited, study in this order:

1. **Energy-pruning problem** - understand every line and especially state restoration.
2. **Hard Q5: pruning + quantization** - strongest algorithmic mock problem.
3. **Hard Q3: shift + rotation + scale + translation** - strongest Fourier-property combination.
4. **MSE-constrained pruning.**
5. **2D CFT separability and signs.**
6. **High-pass, low-pass, notch masks.**
7. **Magnitude threshold + wrapped phase error.**
8. **Previous-year CFT properties.**
9. **NumPy sorting, masks, copies, axis, and broadcasting.**

The recurring pattern to remember is:

understand the mathematical transformation

then

modify only the necessary part of the supplied framework

then

verify numerically using the requested metric .

71. Final Self-Test Checklist

Before the exam, make sure you can answer these without notes:

- Why are there $2N + 1$ harmonics?
- Why does n run from $-N$ to N ?
- Why is c_n computed with a negative exponent?
- Why does reconstruction use a positive exponent?
- What is $|c_n|^2$?
- Why does `np.argsort(...)[-1]` help with pruning?
- How do you map a sorted list index back to harmonic n ?
- Why must each pruning experiment use fresh coefficients?
- What is the difference between $f(t - t_0)$ and $f(t) + B$?
- Why does translation only modify c_0 ?
- Why does differentiation multiply by $j n \omega_0$?
- Why do high harmonics matter more after differentiation?
- Why is phase meaningless near zero magnitude?
- Why must phase error be wrapped?
- What does `axis=-1` mean in `np.trapezoid`?
- Why is an image indexed as `I[y, x]`?
- Why are edges high spatial frequency?
- Why does high-pass filtering reveal edges?
- Why is separability necessary?
- How do low-pass, high-pass, and notch masks differ?
- Why must a quantization/search problem restore original coefficients for every candidate?

If you can answer all of those and implement the minimal templates in Section 68, you are covering the main reasoning and coding patterns exposed by the assignment and previous-year questions.